Locks: ReentrantLock, ReadWriteLock and Conditions

8 min read · Updated 28 Jul 2026 · Topic 39 of 65

Bookmark · Mark done

Why it matters

`synchronized` handles most cases, so the interesting question is what the explicit `Lock` API buys you — and the honest answer is not "speed". It is capabilities `synchronized` cannot express: giving up if the lock is unavailable, waiting with a timeout, being interruptible, and having several wait-sets on one lock. Knowing *which* capability you need is what makes this a design question rather than a syntax one.

The concept, simply

`java.util.concurrent.locks.Lock` is an interface whose main implementation, `ReentrantLock`, does everything `synchronized` does and more — at the cost of manual unlocking.

	`synchronized`	`ReentrantLock`
Release	Automatic, even on exception	**Manual** — must be in `finally`
Try without blocking	No	`tryLock()`
Timeout	No	`tryLock(t, unit)`
Interruptible	No	`lockInterruptibly()`
Fairness	No	Optional FIFO
Wait sets	One per object	Many via `newCondition()`

Lock across methods	No	Yes

The rule of thumb: **use `synchronized` unless you need one of the rows above.** It is shorter, impossible to leak, and the JVM optimises it well.

How it works

The `finally` is not optional. A `Lock` not released in a `finally` block will leak on any exception, and every thread that touches it afterwards hangs forever. This is the one real disadvantage against `synchronized`.

`ReadWriteLock` allows many concurrent readers *or* one writer. It only pays off when reads massively outnumber writes and the critical sections are long enough to matter — otherwise its bookkeeping costs more than a plain lock. `StampedLock` (Java 8) goes further with an optimistic read mode that takes no lock at all and validates afterwards, but it is not reentrant and is easy to misuse.

`Condition` replaces `wait`/`notify` and fixes their biggest weakness: one object has a single wait-set, so `notifyAll` wakes everyone including threads waiting for a different reason. A lock can have several conditions, so you can wake exactly the right group.

Code examples

JAVA

Copy

```java
private final ReentrantLock lock = new ReentrantLock();

public void update() {
    lock.lock();
    try {                       // ✓ the try must be the FIRST thing after
        mutateState();
    } finally {
        lock.unlock();          // ✓ non-negotiable
    }
}
```

```
    if (!lock.tryLock()) return false;
    try { mutateState(); return true; }
    finally { lock.unlock(); }
}

// Timeout — bounded waiting, so a stuck lock does not stall a request thr
public boolean updateWithin(Duration d) throws InterruptedException {
    if (!lock.tryLock(d.toMillis(), TimeUnit.MILLISECONDS)) return false;
    try { mutateState(); return true; }
    finally { lock.unlock(); }
}
```

The bounded buffer, which is the textbook `Condition` example and shows why two conditions beat one:

JAVA

Copy

```
public class BoundedBuffer<T> {
    private final Queue<T> queue = new ArrayDeque<>();
    private final int capacity;
    private final ReentrantLock lock = new ReentrantLock();
    private final Condition notFull  = lock.newCondition();   // two separ
    private final Condition notEmpty = lock.newCondition();   // wait-sets

    public void put(T item) throws InterruptedException {
        lock.lock();
        try {
            while (queue.size() == capacity) notFull.await();   // ✓ WHILE
            queue.add(item);
            notEmpty.signal();          // wake ONLY a consumer
        } finally { lock.unlock(); }
    }

    public T take() throws InterruptedException {
        lock.lock();
        try {
            while (queue.isEmpty()) notEmpty.await();
            T item = queue.poll();
            notFull.signal();           // wake ONLY a producer
```

```
}
// With wait/notify there is ONE wait-set, so notifyAll wakes producers an
// consumers alike and most of them immediately go back to sleep. Worse,
// notify() can wake the wrong kind and deadlock the buffer.
//
// In real code: just use ArrayBlockingQueue, which is exactly this, teste
```

Why the loop must be `while`, never `if`:

JAVA Copy

```
// ✗ if (queue.isEmpty()) notEmpty.await();
// Two reasons it is wrong:
//  1. SPURIOUS WAKEUPS — await() may return without any signal. The JLS
//     permits it, and real JVMs do it.
//  2. Another thread may consume the item between the signal and this
//     thread reacquiring the lock.
// Always re-check the condition in a loop after waking.
```

`ReadWriteLock`, and the honest caveat:

JAVA Copy

```
private final ReadWriteLock rw = new ReentrantReadWriteLock();

public String get(String key) {
    rw.readLock().lock();               // many readers concurrently
    try { return map.get(key); }
    finally { rw.readLock().unlock(); }
}

public void put(String key, String value) {
    rw.writeLock().lock();              // exclusive
    try { map.put(key, value); }
    finally { rw.writeLock().unlock(); }
}
// ⚠ For a map specifically, ConcurrentHashMap beats this comfortably —
//   it locks per bin and never locks reads at all (Chapter 4). ReadWriteL
```

```
//   earns its keep for longer read sections over a structure with no
//   writers. Equivalent.
// ⚠ You cannot upgrade a read lock to a write lock — it deadlocks. Releas
//   and reacquire, then re-check the state.
```

Under the hood

Nearly everything in `java.util.concurrent.locks` is built on `AbstractQueuedSynchronizer` (AQS) — Doug Lea's framework holding an atomic `int` state plus a FIFO queue of waiting threads. `ReentrantLock` uses the state as a hold count, `Semaphore` as permits, `CountDownLatch` as the count. Naming AQS is a strong senior signal because it explains why these classes behave so consistently.

Fairness is expensive. A fair lock hands ownership to the longest-waiting thread, which prevents starvation but forces a context switch on nearly every handoff — throughput can drop by an order of magnitude. The default is unfair, and unfair is almost always right: barging threads keep the CPU busy.

The other AQS-based tools are worth knowing by name, because reaching for the right one beats hand-rolling with locks: `CountDownLatch` (wait for N events, single-use), `CyclicBarrier` (N threads meet, reusable), `Semaphore` (limit concurrent access to N), `Phaser` (flexible multi-phase). "Wait until all workers finish" is a `CountDownLatch`, not a `wait`/`notify` puzzle.

Common mistakes

- **No `finally` around `unlock()`.** One exception leaks the lock and hangs every later thread.
- **Code between `lock()` and `try`.** If it throws, `finally` unlocks a lock you never acquired — `IllegalMonitorStateException`.
- **`if` instead of `while` around `await()`.** Spurious wakeups are real.
- **Using a lock where `synchronized` would do.** More code, more failure modes, no benefit.
- **Fair locks by default.** Large throughput cost for a guarantee you rarely need.
- **Upgrading a read lock to a write lock.** Deadlocks — release first.

Interview lens

Q: "`synchronized` or `ReentrantLock`?"
A: `synchronized` by default — it is shorter, releases automatically even on exception, and the JVM optimises it well. I reach for `ReentrantLock` when I need something it cannot express: `tryLock` to avoid blocking, a timeout, interruptible acquisition, fairness, multiple condition wait-sets, or locking that spans methods. Performance is not the reason; modern `synchronized` is competitive.

Q: "What is the danger of `ReentrantLock`?"
A: Manual release. If `unlock()` is not in a `finally`, any exception leaks the lock and every thread that needs it afterwards blocks forever — a hung application with no exception in the logs. The idiom is `lock()`, then `try` immediately, with `unlock()` in `finally`; putting anything between `lock()` and `try` creates a second bug.

Q: "Why `while` rather than `if` around `await()`?"
A: Two reasons. Spurious wakeups — `await()` is permitted to return without any signal, and real JVMs do it. And even after a genuine signal, another thread may change the state before this one reacquires the lock. So the condition must be re-checked in a loop; the same applies to `Object.wait()`.

Q: "What does `Condition` give you over `wait`/`notify`?"
A: Multiple wait-sets on one lock. With intrinsic monitors there is a single wait-set, so `notifyAll` wakes every waiter regardless of what they were waiting for, and `notify` can wake the wrong one and stall the system. With conditions, a bounded buffer can have `notFull` and `notEmpty` and signal exactly the right group.

A: `AbstractQueuedSynchronizer`, the framework underneath most of `java.util.concurrent`. It holds an atomic int state and a FIFO queue of blocked threads, and each synchroniser interprets the state differently — a hold count for `ReentrantLock`, permits for `Semaphore`, a countdown for `CountDownLatch`. It is why those classes share such consistent behaviour around fairness and interruption.

Q: "When would you use a `ReadWriteLock`?"
A: When reads vastly outnumber writes and the read sections are long enough that concurrency among readers actually pays. For a map I would use `ConcurrentHashMap` instead, which locks per bin and never locks reads. And you cannot upgrade a read lock to a write lock — attempting it deadlocks, so you must release and reacquire, then re-check the state you read.

Cheat-card

- Default to `synchronized`. Use `Lock` for **tryLock · timeout · interruptible · fairness · multiple conditions · cross-method**.
- `lock(); try { … } finally { unlock(); }` — nothing between `lock()` and `try`.
- `await()` always inside a `while` — spurious wakeups are real.
- `Condition` = **multiple wait-sets** per lock (`notFull` / `notEmpty`).
- Fair locks: no starvation, **much lower throughput**. Default unfair.
- `ReadWriteLock`: many readers OR one writer; **no upgrade**. Prefer `ConcurrentHashMap` for maps.
- All built on **AQS** — also `Semaphore`, `CountDownLatch`, `CyclicBarrier`, `Phaser`.
- "Wait for N tasks" = `CountDownLatch`, not hand-rolled `wait` / `notify`.

QUICK CHECK — no marks, just for you.

- It cannot be used across methods
- It is slower
- It cannot be reentrant
- Manual release — if unlock() is not in a finally block, any exception leaks the lock and every thread needing it afterwards blocks forever

Check answer

2. Why must await() always be called inside a while loop rather than an if?

- Because await() can return without any signal — spurious wakeups are permitted and do happen — and because another thread may change the state before this one reacquires the lock
- Because if statements cannot contain await()
- To avoid deadlock
- Because await() returns a boolean

Check answer

3. What does Condition give you that wait/notify does not?

- Better performance
- Multiple wait-sets on one lock, so a bounded buffer can signal 'not full' and 'not empty' separately instead of waking every waiter
- The ability to wait without holding a lock

Check answer

Previous
volatile and the Java Memory Model

Next
Deadlock, Livelock and Starvation

LEARN MORE

ARTICLES & DOCS

- ReentrantLock — API docs
  docs.oracle.com
- AbstractQueuedSynchronizer — API docs
  docs.oracle.com
- Condition — API docs
  docs.oracle.com

VIDEOS



Java Multithreading: Synchronization, Locks, Executors, Deadlock, CountdownLatch & CompletableFuture
Engineering Digest

Hand-picked references for this lesson.